Mock 1 Medium 02 | Customer CDC and SQL Dashboard

SUBMISSION TEMPLATE

Difficulty	Marks	Tasks	Working budget
Medium	20	5	30 minutes (Medium)

Objective

Apply latest-per-customer change events to a customer snapshot, remove deleted customers and build a Databricks SQL dashboard.

Evidence rules - read before starting

There is NO one-task/one-screenshot mandate and NO maximum screenshot count. Provide as many clear taskwise images as the rubric requires. If both implementation and result are assessed, include BOTH. A long SQL/PySpark/pipeline definition may be split into Code_A, Code_B and Code_C screenshots, plus separate Result_A / Result_B screenshots. Images must be legible, not tiny stitched collages. A UI/configuration screenshot is acceptable when the task is menu-based. Never rerun a mutation only to capture evidence; use the original executed code and saved result. The evaluator awards marks only from the submitted visible evidence; merely supplying a workspace link is not a substitute.

Environment and required names

Use your existing assigned Unity Catalog. Reuse schema mock1 for all four questions. Create the schema if missing; do not create a catalog, storage credential, external location, or an unrelated warehouse. Each question creates ONLY its own EXTERNAL volume using the instructor-supplied complete ADLS URI beneath the approved external location. Upload source CSVs inside input/. Managed tables and SDP output must be in schema-managed Delta storage, not inside volume files.

Item	Required value
Catalog	Existing assigned trainee catalog
Schema	mock1
Question external volume	m1_m02_files
Assigned relative ADLS path	mock1/m02/files/ under trainee root
Input directory	/Volumes/<trainee_catalog>/mock1/m1_m02_files/input/
Required saved object(s)	m1_m02_customer_current, m1_m02_customer_changes, m1_m02_customer_dashboard

Key terms and formulas

- I = insert, U = update, D = delete.

- Latest event is the row with the greatest event_sequence for each customer_id.

- Current customers: rows present after change processing; do not invent an is_active column.

Dataset description

File(s): customers_snapshot.csv, customer_changes.csv. Header present; use the supplied data types. Blank values, if any, represent missing values. Do not rename files.

Column	Type	Meaning
customer_id	STRING	Primary business key in both files
customer_name	STRING	Customer name
region	STRING	Region

loyalty_points	INTEGER	Reward points
operation	STRING	Changes only: I, U, D
event_sequence	INTEGER	Changes only: increasing event ordering

Trainee: ___Soumiya Ayyappan_____ Candidate ID: _____ Catalog: _____

Date: _____ Notebook / SQL editor / pipeline / dashboard link: _____

Task requirements and screenshot placeholders

Task 1. Prepare snapshot and change tables | 4 marks

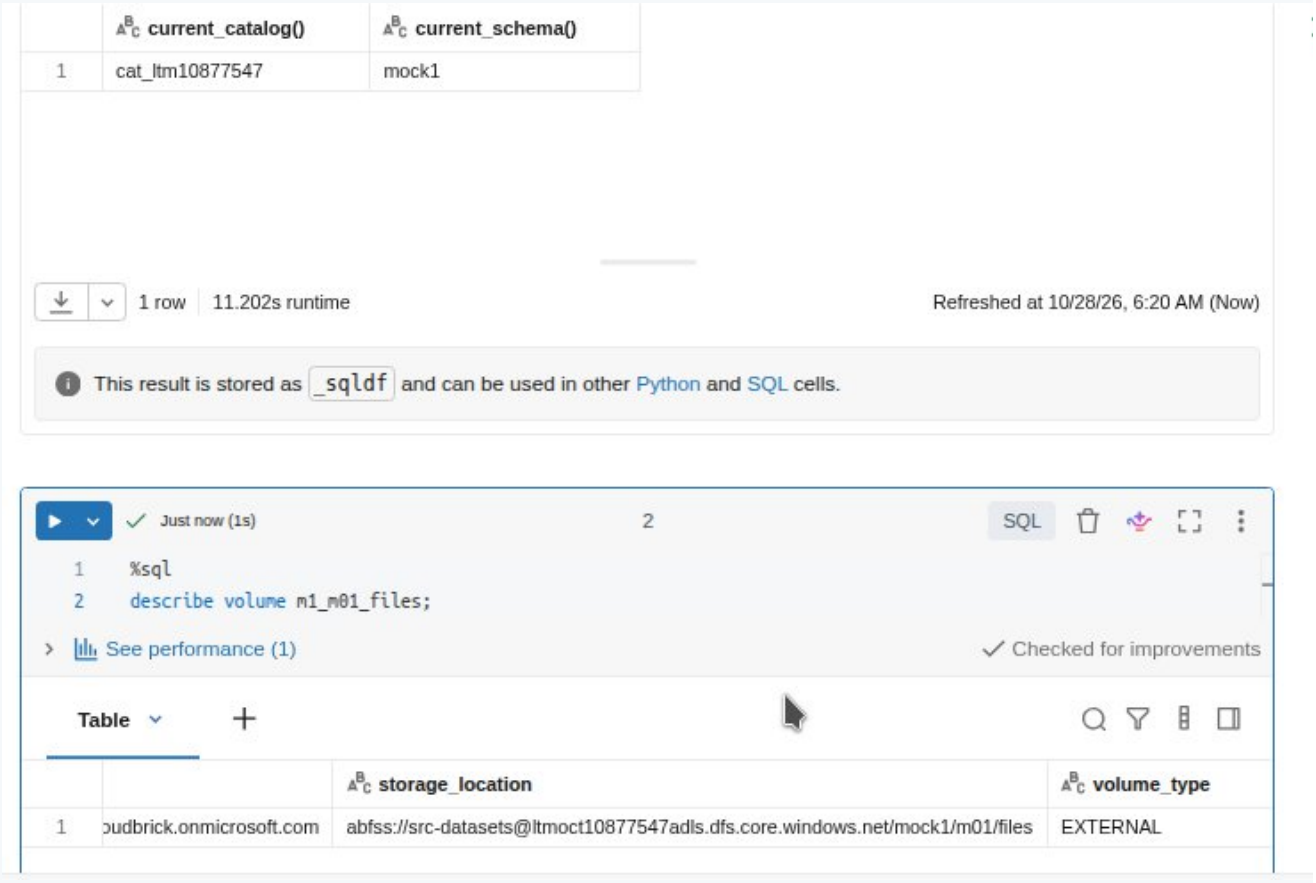
A. Setup UI: volume and both exact filenames.

B. Code: typed reads and both managed table targets.

C. Result: 5 snapshot and 5 change rows, including C101 sequences.

Use descriptive filenames such as M1_M02_T01_Code_A.png, M1_M02_T01_Result_A.png, or M1_M02_T01_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Setup UI: _____



Paste screenshot(s) - Code: _____

06:29 AM (1s) 3 Python

```python
from pyspark.sql.functions import *
from pyspark.sql.types import *
schema=StructType([
    StructField("customer_id",StringType(),True),
    StructField("customer_name",StringType(),True),
    StructField("region",StringType(),True),
    StructField("loyalty_points",IntegerType(),True),

])

df=spark.read.csv("/Volumes/cat_ltm10877547/mock1/m1_m02_files/input/M02_customers_snapshot.csv",header=True,schema=schema)
df.count()
```

See performance (1)

06:31 AM (1s) 4 Python

```python
from pyspark.sql.functions import *
from pyspark.sql.types import *
schema=StructType([
    StructField("customer_id",StringType(),True),
    StructField("customer_name",StringType(),True),
    StructField("region",StringType(),True),
    StructField("loyalty_points",IntegerType(),True),
    StructField("operation",StringType(),True),
    StructField("event_sequence",IntegerType(),True)
])

df2=spark.read.csv("/Volumes/cat_ltm10877547/mock1/m1_m02_files/input/M02_customer_changes.csv",header=True,schema=schema)
df2.count()
```

See performance (1) Checked for

06:34 AM (3s) 5

```python
df.write.format("delta").mode("overwrite").saveAsTable("cat_ltm10877547.mock1.m1_m02_customer_current")
```

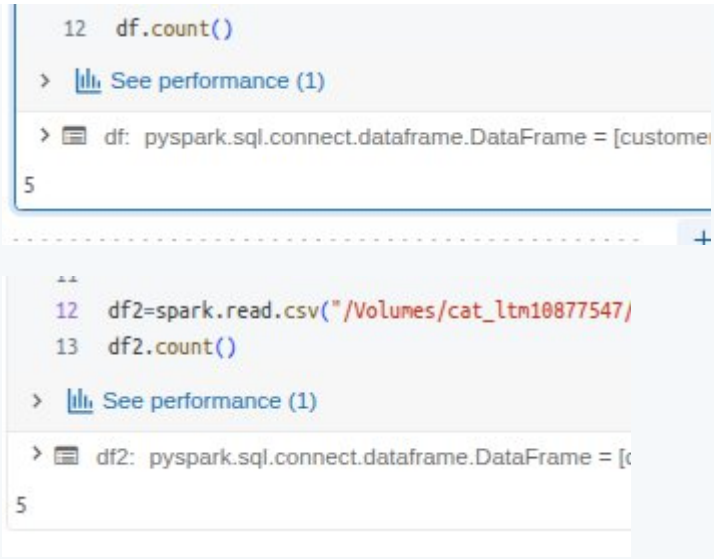
See performance (1) Checked for improvements

06:34 AM (3s) 6

```python
df2.write.format("delta").mode("overwrite").saveAsTable("cat_ltm10877547.mock1.m1_m02_customer_changes")
```

See performance (1) Checked for improvements

Paste screenshot(s) - Result: _____



Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

Task 2. Apply latest insert/update changes | 4 marks

A. Code: deduplication window/rank, key, update branch and insert branch.

B. Result: changed C101, C103, inserted C106 and intermediate row count 6.

Use descriptive filenames such as M1_M02_T02_Code_A.png, M1_M02_T02_Result_A.png, or M1_M02_T02_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Code: _____

```
spark.sql("""
MERGE INTO cat_ltm10877547.mock1.m1_m02_customer_currentt AS target
USING (
  SELECT customer_id, customer_name, region, loyalty_points, operation
  FROM (
    SELECT *,
      ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY event_sequence DESC) AS rn
    FROM cat_ltm10877547.mock1.m1_m02_customer_changess
  )
  WHERE rn = 1
) AS source
ON target.customer_id = source.customer_id
WHEN MATCHED AND source.operation = 'UPDATE' THEN
  UPDATE SET
    target.customer_name = source.customer_name,
    target.region = source.region,
    target.loyalty_points = source.loyalty_points
WHEN NOT MATCHED THEN
  INSERT (customer_id, customer_name, region, loyalty_points)
  VALUES (source.customer_id, source.customer_name, source.region, source.loyalty_points)
""")

# Verify results
result_df = spark.table("cat_ltm10877547.mock1.m1_m02_customer_currentt")
print(f"Row count after MERGE: {result_df.count()}")
```

Paste screenshot(s) - Result: _____

```
result_df:  pyspark.sql.connect.dataframe.DataFrame = [custome
Row count after MERGE: 6
+-----------+-------------+------+--------------+
|customer_id|customer_name|region|loyalty_points|
+-----------+-------------+------+--------------+
|       C101|         Asha| North|           100|
|       C102|         Bala| South|           200|
|       C103|       Chitra| North|           150|
|       C104|          Dev|  West|            80|
|       C105|         Esha| South|           120|
|       C106|        Farah|  West|            90|
+-----------+-------------+------+--------------+
```

Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

Task 3. Apply delete event | 4 marks

A. Code: latest D event selection and targeted deletion.

B. Result: C102 absent and remaining five customer IDs displayed.

Use descriptive filenames such as M1_M02_T03_Code_A.png, M1_M02_T03_Result_A.png, or M1_M02_T03_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Code: _____

```
from pyspark.sql.window import Window

delete_df = (
    spark.table("cat_ltm10877547.mock1.m1_m02_customer_changess")
    .filter(col("operation") == "D")
    .withColumn("rn", row_number().over(
        Window.partitionBy("customer_id").orderBy(col("event_sequence").desc())
    ))
    .filter(col("rn") == 1)
    .select("customer_id")
)

delete_ids = [row.customer_id for row in delete_df.collect()]
print(f"Customer IDs marked for DELETE: {delete_ids}")

# Perform targeted deletion from the current table
spark.sql(f"""
    DELETE FROM cat_ltm10877547.mock1.m1_m02_customer_currentt
    WHERE customer_id IN ({','.join(f"'{cid}'" for cid in delete_ids)})
""")

# B. Verify C102 is absent and display remaining customer IDs
result_df = spark.table("cat_ltm10877547.mock1.m1_m02_customer_currentt").orderBy("customer_id")
print(f"Row count after DELETE: {result_df.count()}")
result_df.show()
```

Paste screenshot(s) - Result: _____

```
> 2 dataframes: delete_df, result_df
Customer IDs marked for DELETE: ['C102']
Row count after DELETE: 5
+-----------+-------------+------+--------------+
|customer_id|customer_name|region|loyalty_points|
+-----------+-------------+------+--------------+
|       C101|         Asha| North|           100|
|       C103|       Chitra| North|           150|
|       C104|          Dev|  West|            80|
|       C105|         Esha| South|           120|
|       C106|        Farah|  West|            90|
+-----------+-------------+------+--------------+
```

Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

Task 4. Reconcile final regional loyalty metrics | 4 marks

A. Code: aggregate saved current table with region and SUM(loyalty_points).

B. Result: three region rows and grand total 5 / 600.

Use descriptive filenames such as M1_M02_T04_Code_A.png, M1_M02_T04_Result_A.png, or M1_M02_T04_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Code: _____

```
2
3  spark.sql("select count(*),sum(loyalty_points) from dbt_ltm10877547.mock1.m1_m02_customer_currentt ").show()
```

See performance (1)

Paste screenshot(s) - Result: _____

```
+--------+---+
|count(*)|sum|
+--------+---+
|       5|600|
+--------+---+
```

Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

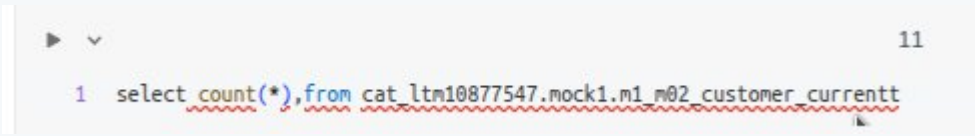
Task 5. Build Databricks SQL dashboard | 4 marks

A. Code/query editor: source aggregation and KPI queries or visible query definitions.

B. Dashboard result: readable region bars with values and current-customer KPI = 5.

Use descriptive filenames such as M1_M02_T05_Code_A.png, M1_M02_T05_Result_A.png, or M1_M02_T05_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Code/query editor: _____



Paste screenshot(s) - Dashboard result: _____



Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

Evaluation rubric and evidence map

Task	Criterion	Marks	Evidence
1	Correct external inputs and two Delta tables	2	Code / UI / Result as specified in Task 1
1	Correct source counts and sequencing evidence	2	Code / UI / Result as specified in Task 1
2	Latest event selection correct; no stale C101 update	2	Code / UI / Result as specified in Task 2

2	Correct MERGE implementation and changed rows	2	Code / UI / Result as specified in Task 2
3	Correct latest-event D logic and targeted DELETE	2	Code / UI / Result as specified in Task 3
3	Correct five-row post-delete state	2	Code / UI / Result as specified in Task 3
4	Correct final current-table aggregation	2	Code / UI / Result as specified in Task 4
4	Correct region and overall totals	2	Code / UI / Result as specified in Task 4
5	Dashboard datasets based on final saved table	2	Code / UI / Result as specified in Task 5
5	Correct bar chart and KPI output	2	Code / UI / Result as specified in Task 5

Total marks: 20. Task count: 5. Screenshot count: variable; provide enough legible evidence to cover every rubric criterion.

Submission requirements

Complete the matching submission template with task-specific screenshot(s) showing every assessed code/configuration and output item. Multiple screenshots per task are allowed and sometimes necessary. Code-only evidence does not prove output; result-only evidence does not prove implementation when the rubric assesses both. Do not submit a generic solution script in place of evidence. Keep your original notebook, Delta objects, pipeline, dashboard and version history accessible. No extra hidden tests or additional report are required.